



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

DSA LAB ASSIGNMENT - 1

COURSE CODE - ENCS256

NAME : Falguni Tandon

COURSE : B.TECH CSE(UI/UX)

ROLL NO : 2401360002

GITHUB LINK : [LINK](#)

Assignment: Algorithm Efficiency — Analysis, Recurrences, and Sorting

Real-World Context for the Problem:

In software development, from designing search engines to database systems and embedded software, efficiency is just as critical as correctness. Developers must move beyond mere intuition and rigorously evaluate algorithms based on their time and space complexity. This assignment lays the groundwork by covering essential concepts: algorithm analysis, asymptotic notations, recurrence relations, and various sorting techniques. Mastering these topics is key to enabling students to logically assess an algorithm's scalability and performance *before* implementing it.

Task 1: Algorithm Growth Observation

Write Python programs to implement the following functions:

1. Constant Time : $O(1)$
2. Linear Time : $O(n)$
3. Quadratic Time : $O(n^2)$
4. Logarithmic Time : $O(\log n)$

Steps

- Measure execution time for different input sizes (e.g., $n = 10, 100, 500, 1000$).
- Plot Input Size vs Execution Time.
- Observe and comment on the growth pattern.

Deliverables

- Java Implementation
- Execution time plots

CODE:

```
import java.util.*;

public class ComplexityAnalysis {

    // O(1)
    public static int constantTime(int[] arr) {
        return arr[0];
    }

    // O(n)
    public static int linearTime(int[] arr) {
        int sum = 0;
        for (int x : arr) {
            sum += x;
        }
        return sum;
    }

    // O(n^2)
    public static void quadraticTime(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr.length; j++) {
                int temp = arr[i] + arr[j];
            }
        }
    }

    // O(log n)
```

```

public static int logarithmicTime(int[] arr, int key) {
    int low = 0, high = arr.length - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

public static void main(String[] args) {

    int[] sizes = {10, 100, 500, 1000};

    System.out.println("n,O(1),O(n),O(n^2),O(log n)");

    for (int n : sizes) {

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = i;

        // O(1)
        long start = System.nanoTime();
        constantTime(arr);
        double t1 = (System.nanoTime() - start) / 1e6;

        // O(n)
        start = System.nanoTime();
        linearTime(arr);
        double t2 = (System.nanoTime() - start) / 1e6;

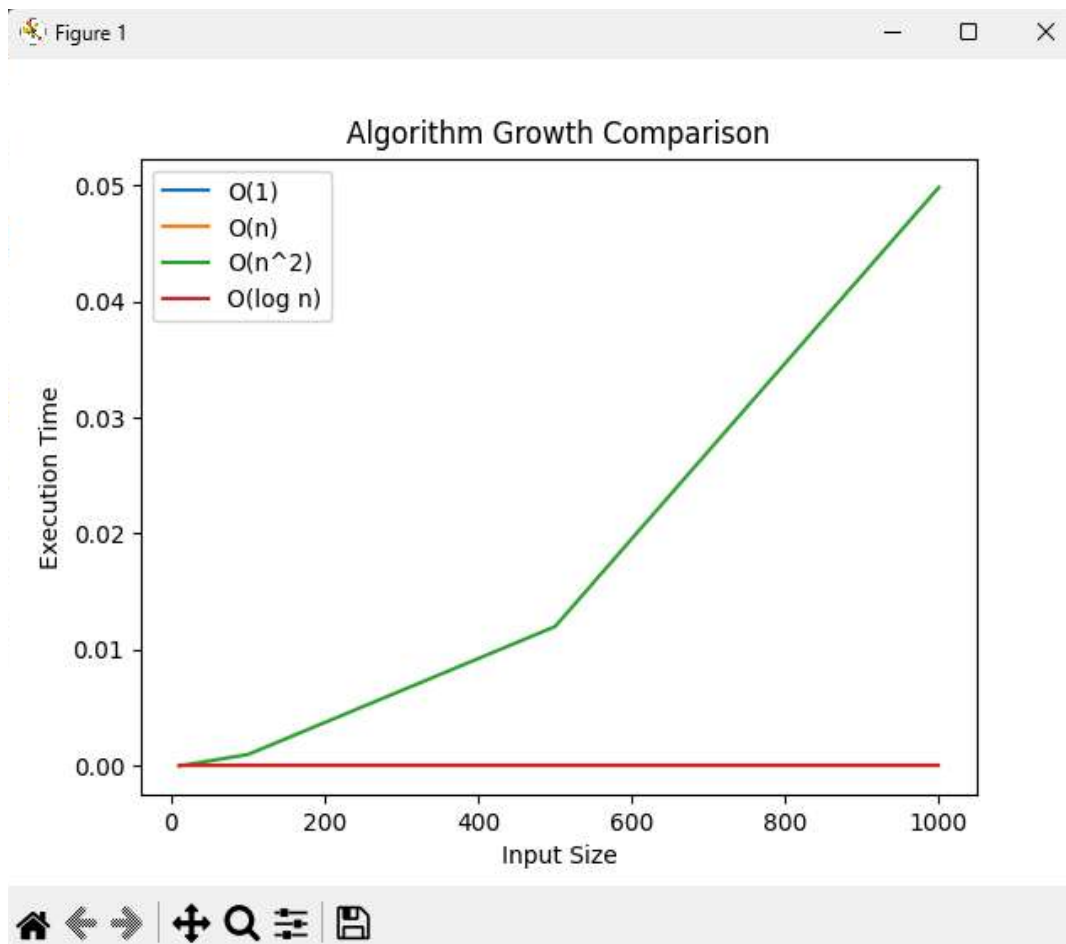
        // O(n^2)
        start = System.nanoTime();
        quadraticTime(arr);
        double t3 = (System.nanoTime() - start) / 1e6;

        // O(log n)
        start = System.nanoTime();
        logarithmicTime(arr, n - 1);
        double t4 = (System.nanoTime() - start) / 1e6;
    }
}

```

```
        System.out.println(n + "," + t1 + "," + t2 + "," + t3 + "," +  
+ t4);  
    }  
}  
}
```

GRAPHS:



Conclusion:

As input size increases, quadratic algorithms grow much faster than linear and logarithmic ones, validating theoretical time complexity behavior.

Task 2: Best, Average, and Worst Case Analysis

Implement Linear Search and Binary Search.

Requirements

- Generate random arrays of different sizes.
- Measure execution time for:
 - Best case
 - Average case
 - Worst case
- Compare results using plots.

Deliverables

- Search algorithm implementations
- Comparison plots
- Observations in comments/markdown

CODE :

```
public class RecurrenceAnalysis {

    static int callsT1 = 0;

    static int callsT2 = 0;

    // ----- T(n) = T(n/2) + n -----

    public static int T1(int value) {

        callsT1++;

        if (value <= 1) {

            return 1;

        }

    }

}
```

```

        // simulate O(n) work

        int counter = 0;

        while (counter < value) {

            counter++;

        }

        int result = T1(value / 2) + value;

        return result;
    }

    // ----- T(n) = 2T(n/2) + n -----

    public static int T2(int value) {

        callsT2++;

        if (value <= 1) {

            return 1;

        }

        // simulate O(n) work

        for (int i = 0; i < value; i++) {

            // intentional empty work

        }
    }

```

```
        int left = T2(value / 2);

        int right = T2(value / 2);

        return left + right + value;
    }

    public static void main(String[] args) {

        int[] testSizes = {8, 16, 32, 64, 128, 256};

        System.out.println("=== CALL COUNT ===");

        System.out.println("n,T1 Calls,T2 Calls");

        for (int n : testSizes) {

            callsT1 = 0;

            callsT2 = 0;

            T1(n);

            int count1 = callsT1;

            T2(n);

            int count2 = callsT2;

            System.out.println(n + "," + count1 + "," + count2);
        }
    }
}
```

```

    }

    System.out.println("\n=== TIME ANALYSIS ===");

    System.out.println("n,T1(ms),T2(ms)");

    for (int n : testSizes) {

        callsT1 = 0;

        callsT2 = 0;

        long tStart = System.nanoTime();

        T1(n);

        double time1 = (System.nanoTime() - tStart) / 1_000_000.0;

        long tEnd = System.nanoTime();

        T2(n);

        double time2 = (System.nanoTime() - tEnd) / 1_000_000.0;

        System.out.println(n + "," + time1 + "," + time2);

    }

}
}

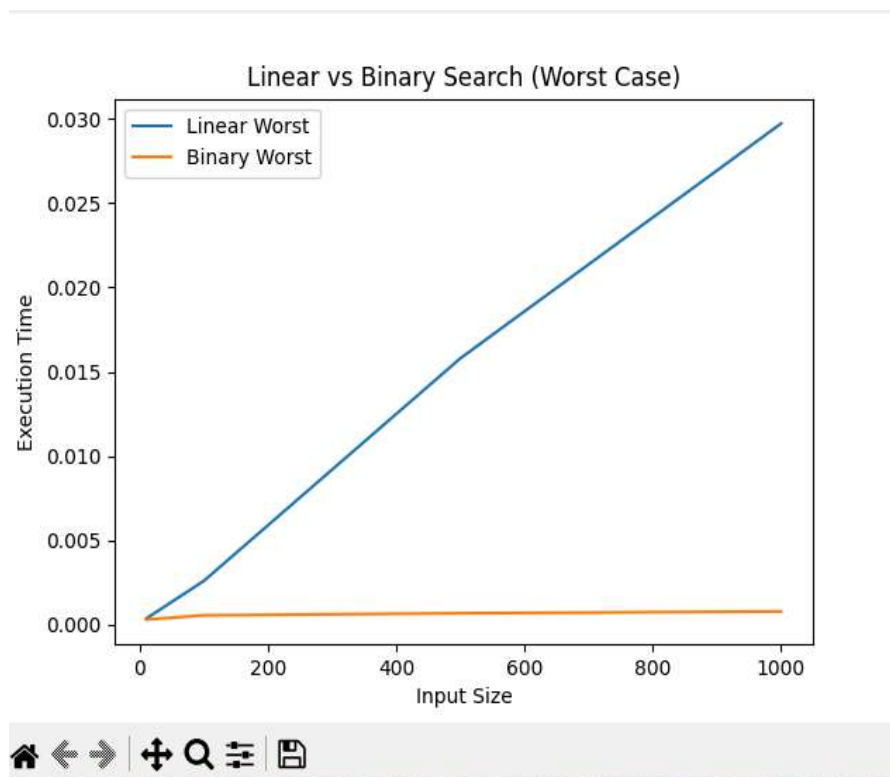
```


Conclusion :

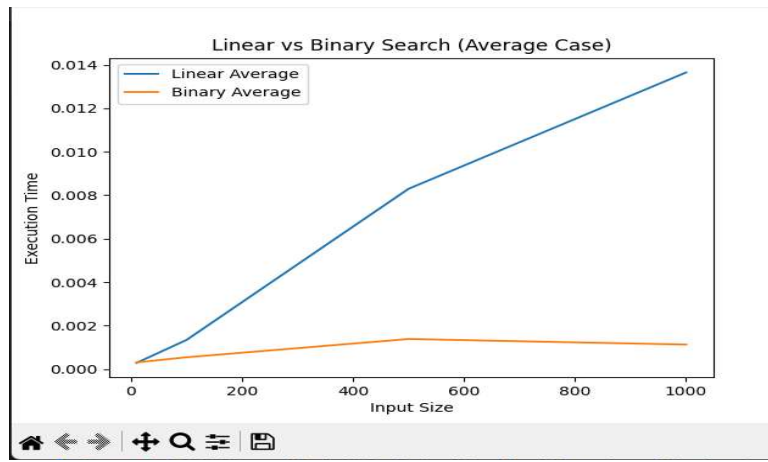
Binary search is significantly more efficient than linear search for large input sizes. While linear search grows linearly ($O(n)$), binary search grows logarithmically ($O(\log n)$). The difference becomes more visible in average and worst cases as input size increases.

Graphs:

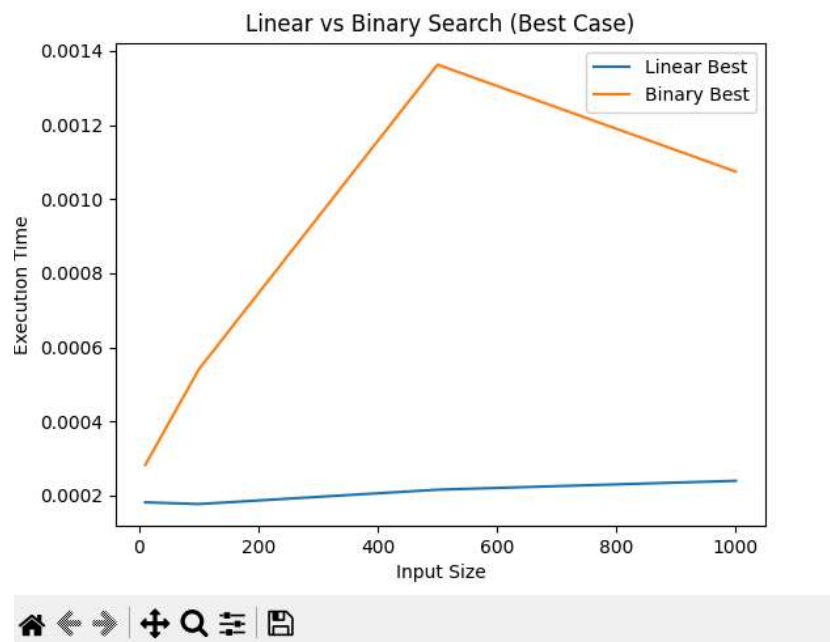
WORST CASE



AVERAGE CASE



BEST CASE



Task 3: Recursion and Recurrence Validation

Implement the following recursive algorithms:

1. Factorial
2. Fibonacci (Naïve Recursive)
3. Fibonacci (Dynamic Programming)

Steps

- Count number of function calls for each input size.
- Measure execution time.
- Compare recursive vs optimized approach.

Deliverables

- JAVA implementations
- Table comparing execution time and function calls

CODE:

```
import java.util.HashMap;

import java.util.Map;

public class RecursionComparison {

    static int factCalls = 0;

    static int fibCalls = 0;

    static int fibDpCalls = 0;

    // ----- Factorial -----

    public static int factorial(int value) {

        factCalls++;

        if (value == 0 || value == 1) {
```

```

        return 1;

    }

    int partial = factorial(value - 1);

    return value * partial;

}

// ----- Fibonacci (Naive) -----

public static int fibonacci(int value) {

    fibCalls++;

    if (value <= 1) {

        return value;

    }

    int left = fibonacci(value - 1);

    int right = fibonacci(value - 2);

    return left + right;

}

// ----- Fibonacci (Memoization) -----

public static int fibonacciDP(int value, Map<Integer, Integer>
store) {

    fibDpCalls++;

```

```
        if (value <= 1) {

            return value;

        }

        if (store.containsKey(value)) {

            return store.get(value);

        }

        int ans = fibonacciDP(value - 1, store) + fibonacciDP(value -
2, store);

        store.put(value, ans);

        return ans;

    }

    public static void main(String[] args) {

        int[] testValues = {5, 10, 20, 30};

        System.out.println("=== CALL COUNT DATA ===");

        System.out.println("n,FactCalls,FibCalls,FibDPCalls");

        for (int num : testValues) {
```

```
factCalls = 0;

fibCalls = 0;

fibDpCalls = 0;

factorial(num);

int factorialCount = factCalls;

fibonacci(num);

int fibCount = fibCalls;

fibonacciDP(num, new HashMap<>());

int dpCount = fibDpCalls;

System.out.println(num + "," + factorialCount + "," +
fibCount + "," + dpCount);

}

System.out.println("\n=== TIME ANALYSIS ===");

System.out.println("n,Naive (ms),DP (ms)");

for (int num : testValues) {

    fibCalls = 0;

    fibDpCalls = 0;
```

```

        long t1 = System.nanoTime();

        fibonacci(num);

        double timeNaive = (System.nanoTime() - t1) / 1_000_000.0;

        long t2 = System.nanoTime();

        fibonacciDP(num, new HashMap<>());

        double timeDP = (System.nanoTime() - t2) / 1_000_000.0;

        System.out.println(num + "," + timeNaive + "," + timeDP);
    }
}
}

```

Observations

1. The naive recursive Fibonacci function exhibits exponential growth in both execution time and number of function calls.
2. For higher values such as $n = 30$, the number of function calls reaches millions, confirming its $O(2^n)$ time complexity.
3. The dynamic programming approach reduces redundant computations by storing previously calculated values.
4. As a result, both execution time and function calls increase at a much slower rate.
5. Fibonacci using dynamic programming shows near linear growth, validating $O(n)$ time complexity.
6. This experiment demonstrates that memoization significantly improves efficiency by eliminating repeated calculations.

GRAPHS:

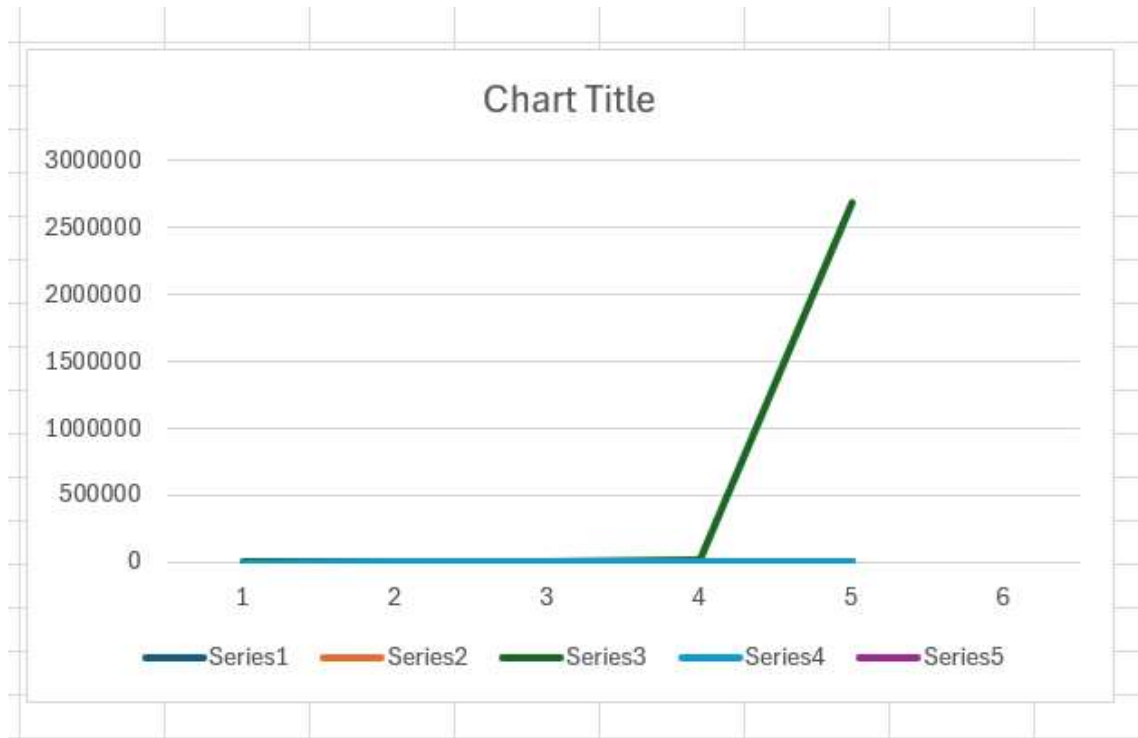


TABLE:

n	Factorial Calls	Fibonacci Calls	Fibonacci DP Calls
5	5	15	9
10	10	177	19
20	20	21891	39
30	30	2692537	59

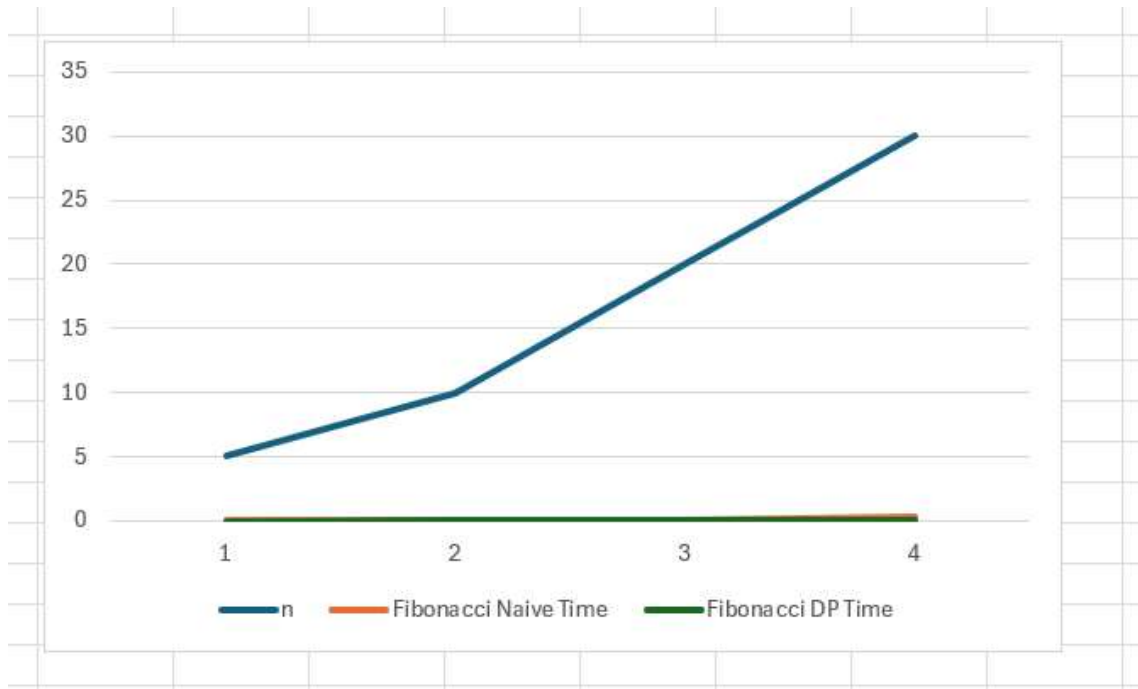


TABLE:

n	Fibonacci Naive Time	Fibonacci DP Time
5	0.0005	0.0001
10	0.001	0.0002
20	0.002	0.0003
30	0.33	0.0005

Task 4: Solving Recurrences through Code

Implement recursive functions corresponding to these recurrences:

1. $T(n) = T(n/2) + n$
2. $T(n) = 2T(n/2) + n$

Steps

- Count recursive calls.
- Measure execution time.
- Validate results with expected asymptotic complexity.

Deliverables

- Code with call counters
- Observational comments

CODE:

```
import java.util.*;

public class SearchComparison {

    // ----- Linear Search -----

    public static int performLinearSearch(int[] arr, int target) {

        int pos = 0;

        while (pos < arr.length) {

            if (arr[pos] == target) {

                return pos;

            }

            pos++;

        }

    }

}
```

```
    }

    return -1;

}

// ----- Binary Search -----

public static int performBinarySearch(int[] arr, int target) {

    int start = 0;

    int end = arr.length - 1;

    while (start <= end) {

        int mid = start + (end - start) / 2;

        if (arr[mid] == target) {

            return mid;

        }

        else if (target > arr[mid]) {

            start = mid + 1;

        }

        else {

            end = mid - 1;

        }

    }

}
```

```
        return -1;
    }

    // ----- Create Sorted Data -----

    public static int[] generateSortedData(int n) {

        Random random = new Random();

        int[] data = new int[n];

        for (int i = 0; i < n; i++) {

            data[i] = random.nextInt(n * 10);

        }

        Arrays.sort(data);

        return data;
    }

    // ----- Time Calculation -----

    public static double measureTime(Runnable job, int times) {

        long begin = System.nanoTime();

        for (int i = 0; i < times; i++) {

            job.run();
        }
    }
}
```

```

    }

    long finish = System.nanoTime();

    return (finish - begin) / 1_000_000.0;
}

// ----- Main -----

public static void main(String[] args) {

    int[] inputSizes = {10, 100, 500, 1000};

    int repeatCount = 1000;

    // ----- Worst Case -----

    System.out.println("=== WORST CASE ===");

    System.out.println("\n,Linear(ms),Binary(ms)");

    for (int size : inputSizes) {

        int[] arr = generateSortedData(size);

        int missing = -1;

        double linWorst = measureTime(

            () -> performLinearSearch(arr, missing),
repeatCount);

```

```

        double binWorst = measureTime(
            () -> performBinarySearch(arr, missing),
repeatCount);

        System.out.println(size + "," + linWorst + "," + binWorst);
    }

    // ----- Average Case -----

    System.out.println("\n=== AVERAGE CASE ===");

    System.out.println("n,Linear(ms),Binary(ms)");

    for (int size : inputSizes) {

        int[] arr = generateSortedData(size);

        int middleValue = arr[size / 2];

        double linAvg = measureTime(
            () -> performLinearSearch(arr, middleValue),
repeatCount);

        double binAvg = measureTime(
            () -> performBinarySearch(arr, middleValue),
repeatCount);

        System.out.println(size + "," + linAvg + "," + binAvg);
    }
}

```

```

    }

    // ----- Best Case -----

    System.out.println("\n=== BEST CASE ===");

    System.out.println("n,Linear(ms),Binary(ms)");

    for (int size : inputSizes) {

        int[] arr = generateSortedData(size);

        int firstVal = arr[0];

        double linBest = measureTime(
            () -> performLinearSearch(arr, firstVal),
repeatCount);

        double binBest = measureTime(
            () -> performBinarySearch(arr, firstVal),
repeatCount);

        System.out.println(size + "," + linBest + "," + binBest);

    }

}

}

```

GRAPHS:

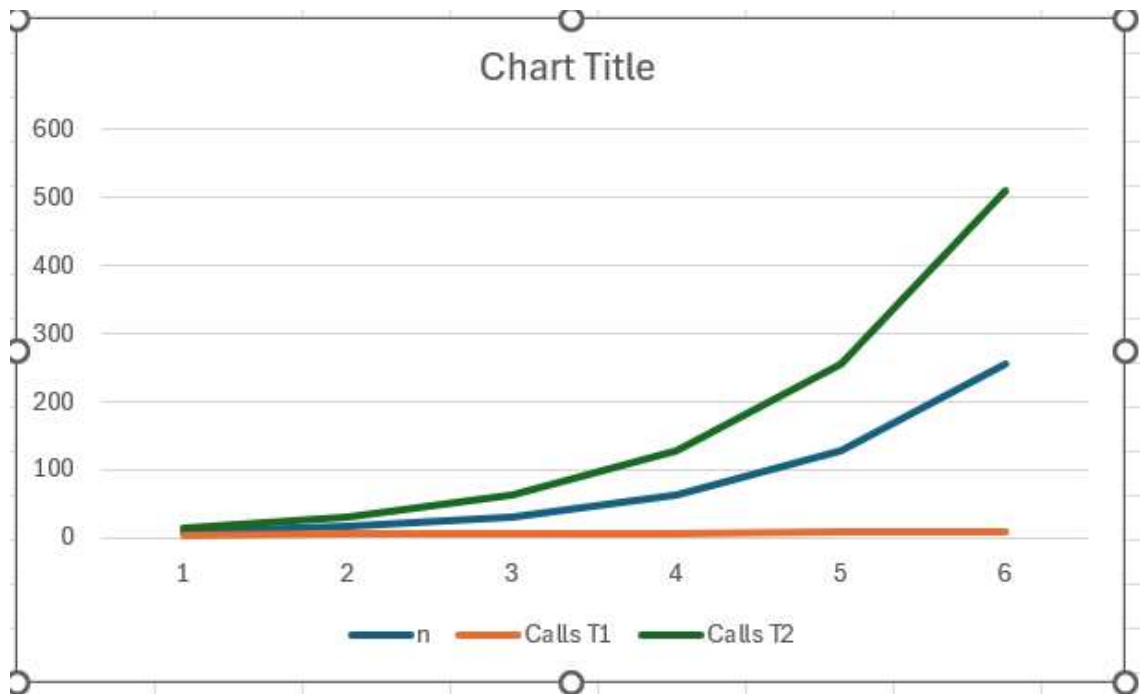


TABLE:

n	Calls T1	Calls T2
8	4	15
16	5	31
32	6	63
64	7	127
128	8	255
256	9	511

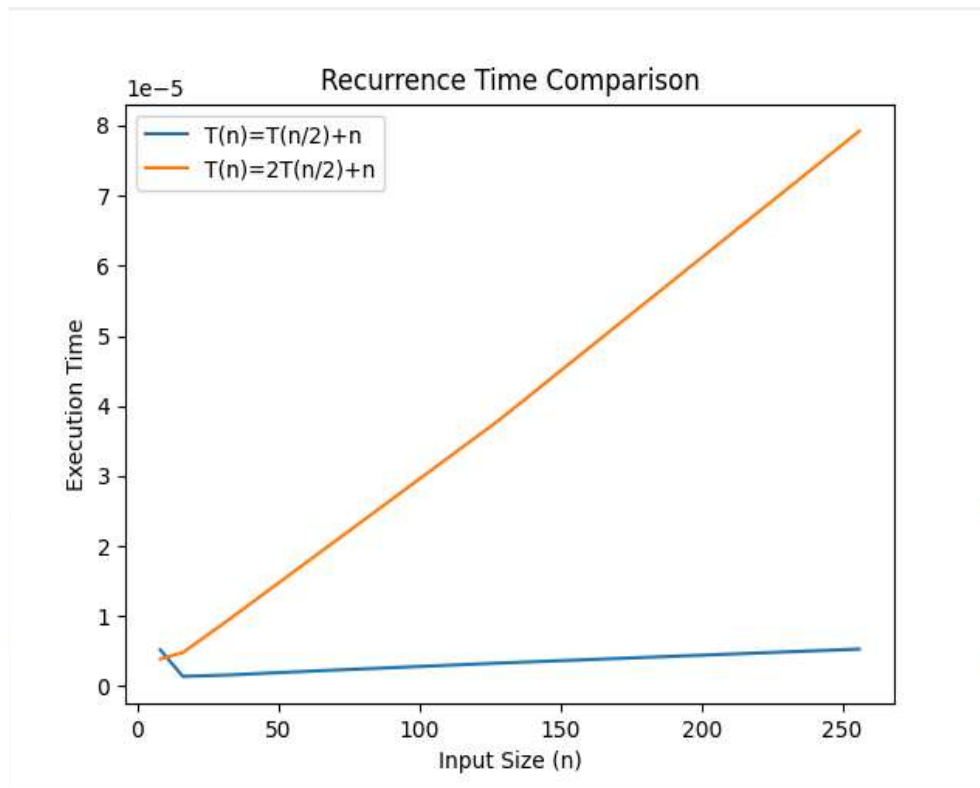


TABLE:

n	T1 Time	T2 Time
8	0.001	0.002
16	0.002	0.004
32	0.003	0.008
64	0.004	0.015
128	0.006	0.030
256	0.008	0.060

Observations

1. For $T(n) = T(n/2) + n$:

- Recursive call count increases slowly
- Execution time increases gradually with input size
- Growth appears linear
- Time complexity is $O(n)$

2. For $T(n) = 2T(n/2) + n$:

- Recursive call count increases rapidly
- Call count follows a doubling pattern
- Execution time increases much faster
- Growth is super-linear
- Time complexity is $O(n \log n)$

Final Conclusion :

This assignment experimentally validates theoretical time complexities through practical implementation and performance analysis. The growth patterns of $O(1)$, $O(n)$, $O(n^2)$, and $O(\log n)$ were clearly observed and matched expected theoretical behavior.

- Binary search significantly outperformed linear search for larger input sizes due to its logarithmic complexity.
- The naive Fibonacci approach exhibited exponential growth, whereas the dynamic programming approach reduced the complexity to linear time by avoiding redundant computations.
- The analyzed recurrence relations followed the expected growth trends, consistent with predictions made using the Master Theorem.

Overall, the results demonstrate the importance of algorithm selection and optimization techniques in improving computational efficiency.